

Decision Log

Interpreting the requirements

- TODO list with multiple users, interpreted as:
 - One or more task board, that can dynamically be created
 - Different roles per user and board (e.g. Admin, Manager and Contributor)
- Concurrent access means:
 - Protecting task edits. Need to reject conflicting edits.
 - Real-time delivery to provide users up to date tasks (deferred as explained below)
- Due dates are calendar days only
 - So dates do not shift across timezones and overdue labels use the viewer's local day
- Recurrence uses RRULE and named timezones
 - An existing standard avoids custom scheduling syntax
- Task dependencies stay within a board and cannot form cycles
 - Self-dependencies and cycles (e.g. Task A <- Task B <- Task A) are rejected
 - We treat archived tasks as settled so cancelled work does not block dependents
- Archived tasks:
 - Tasks can be shown and restored on the same board
- Recurring occurrences are separate tasks
 - Recurring template produces new tasks without copying dependencies or attachments (avoiding blockers or duplicated files)

Architecture and trade-offs

- Modular monolith + React + worker
 - Simpler than microservices, considering the scale of the app at the moment
 - The worker runs independently but shares backend code and data
- PostgreSQL + Prisma
 - Transactions based changes (related changes are saved together safely)
 - But unsupported Prisma constraints require SQL migrations
- Shared Zod contracts + OpenAPI
 - Less validation/documentation drift, at the cost of coupling apps to one contract package
- Better Auth
 - Avoids custom credential/session machinery (can easily plug social login)
 - Requires following the library's schema and contracts
- Redis/BullMQ + S3-compatible storage
 - Supports background jobs, caching and attachments (useful in our use case)
 - But adds operational complexity
- Feature-first frontend folders
 - Keeps workflows together rather than scattered by file type, easier in the current scale
 - If project grows, may require a restructure (feature folders are getting slightly messy)
- Optimistic UI updates
 - Task changes appear immediately, failed requests require rollback and version conflicts need explicit recovery.
 - As a user it feels quick and gives a good UX, TODO management does not have a high risk so optimistic UI makes sense

Scope choices and deferred work

Ordered by proposed priority, in terms of development risk, time estimation and value:

1. Real-time updates across tabs/users
 - Currently version checks prevent overwrites but do not push other users' changes to the screen
2. Bulk operations
 - Individual actions work
3. Sentry-style error reporting
 - Logs, request IDs and health checks exist, would need to add data redaction, source maps and actionable alerts
4. Task change history
 - Versions and timestamps are currently not an audit trail, good to have for “undo” and “blame” feature
5. Task notifications, exact due times
 - Useful for follow-up, but require delivery preferences (email, in-app...) and timezone rules (due date)
6. Nontechnical rich-text editor
 - Markdown controls and preview work but with limited capabilities, switch to a WYSIWYG editing
7. Typo-tolerant search
 - Need to add fuzzy ranking and relevance tuning
8. Custom roles and reporter
 - Add another layer to the database, more API endpoints and UI
9. Feature flags
 - Safety of releasing features, potentially coupled with experimentation
10. MCP capabilities or AI chat
 - Common nowadays with TODO apps, high efficiency increase for the user
 - AI chat is heavy engineering, MCP probably a good first phase

What I would do differently with more time

- Avoid putting optional requirement into the original plan
- Avoid building an overly complex UI with features that were not even in the plan
 - Was fixed along the way but it did make the AI implement and guess features that I did not want to be implemented.
- Validate assumptions earlier.
 - Archive/dependency behavior edge case were caught after implementation but could have easily be noticed earlier
- Improvements through user feedback.
 - Test workflows with nontechnical users before choosing between richer editing, bulk actions and notifications.
- Make collaboration conflicts easier to resolve
 - Show what changed and preserve unsaved input when another user updates the same task
- Verify deployment and rollback procedures
 - Rehearse releases against production-like data, including migration compatibility and recovery from failed deployments
- Harden attachment handling for public use
 - Add malware scanning and review storage quotas, retention rules and cleanup monitoring